

Business Requirements Document (BRD)

Overview

This document outlines the business requirements for listing all APIs in the Code-Classic API Encryption project. The goal is to provide a clear understanding of the existing API endpoints, propose any necessary changes, and outline the next steps for implementation.

Existing System Summary

The current system consists of a RESTful API implemented in Java using Spring Framework. The API provides endpoints for testing, encrypting, and decrypting data. The following endpoints are currently available:

1. **GET /test**
 - **Controller**: MainController
 - **Method**: testModel
 - **Summary**: Returns a simple message indicating that the API is working.
2. **GET /enc**
 - **Controller**: MainController
 - **Method**: encryptModel
 - **Summary**: Encrypts the provided data using a secret key.
3. **GET /dec**
 - **Controller**: MainController
 - **Method**: decryptModel
 - **Summary**: Decrypts the provided encrypted data using a secret key.

Proposed Changes

To enhance the usability and clarity of the API, the following changes are proposed:

1. **Documentation of API Endpoints**: Create comprehensive documentation for each endpoint, including:
 - Request parameters
 - Response formats
 - Error handling
2. **API Versioning**: Implement versioning for the API to ensure backward compatibility in future updates.
3. **Enhanced Security Measures**: Review and implement additional security measures for handling sensitive data, such as rate limiting and input validation.

Business Rules

1. All API endpoints must return a standardized response format, including success and error messages.
2. Sensitive data must be encrypted during transmission and storage.
3. API requests must be authenticated to prevent unauthorized access.

APIs to Consider

- **GET /test**: Simple health check endpoint.
- **GET /enc**: Endpoint for encrypting data.
- **GET /dec**: Endpoint for decrypting data.

Risks & Assumptions

Risks

- Potential security vulnerabilities if encryption and decryption processes are not properly implemented.
- Lack of documentation may lead to misuse of the API by developers.

Assumptions

- The development team has the necessary expertise in implementing security best practices.
- Users of the API will have a basic understanding of encryption and decryption processes.

Next Steps

1. **Documentation**: Assign a technical writer to create detailed API documentation.
2. **Security Review**: Conduct a security assessment of the current encryption methods.
3. **Versioning Plan**: Develop a plan for implementing API versioning.
4. **Stakeholder Review**: Schedule a meeting with stakeholders to review proposed changes and gather feedback.

By following these steps, we can ensure that the API is not only functional but also secure and user-friendly.